Evaluer la qualité d'un modèle

Lino Galiana
Version du 20 octobre 2026

La raison d'être du *machine learning* est de chercher à créer des règles de décision qui ont de bonnes performances prédictives sur un nouvel échantillon. Pour éviter le surapprentissage, c'est-à-dire pour avoir un modèle ayant une bonne validité externe, outre la préparation des données vue dans le chapitre précédent, il sera nécessaire d'évaluer les modèles. Ce chapitre se plonge sur le sujet de l'évaluation des modèles et des enjeux de ceci. Cela permettra d'évoquer les enjeux de l'évaluation dans un cadre d'apprentissage supervisé comme non supervisé, de présenter la méthode de la validation croisée mais aussi d'ouvrir à des concepts comme le *data drift* ou l'évaluation des modèles type LLM à l'état de l'art.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Une méthodologie pour éviter le sur-apprentissage 2
2 Comment évaluer un modèle ? 4
2.1 Apprentissage supervisé 4
2.2 Apprentissage non supervisé 6
2.3 Comment sont évalués les grands modèles de langage et les IA génératives ? 7
2.4 Evaluer sans être tourné vers le passé: les enjeux de la supervision des modèles 7
Informations additionnelles 8

:::: {.content-visible when-profile="fr"}

Compétences à l'issue de ce chapitre

- Comprendre que l'objectif du machine learning est d'élaborer des règles de décision efficaces sur des données nouvelles, en favorisant la validité externe plutôt que le simple ajustement aux données d'entraînement ;
- Mettre en place une méthodologie pour éviter le sur-apprentissage (overfitting), notamment via la séparation rigoureuse entre ensembles d'entraînement et de validation ;
- Connaître les méthodes d'évaluation selon le type d'apprentissage : apprentissage supervisé (classification, régression) et non supervisé ;
- Utiliser la validation croisée pour obtenir une évaluation plus robuste et fiable des performances du modèle ;
- Prendre en compte les enjeux modernes de l'évaluation, comme le data drift (dérive des données) et la supervision continue des modèles en production ;
- Comprendre les nouvelles problématiques liées à l'évaluation des modèles génératifs tels que les grands modèles de langage (LLM).

::::

Le *machine learning* a l'ambition de proposer des méthodes prédictives simples à mettre en oeuvre sur le plan opérationnel. Cette promesse séduit forcément des acteurs ayant un volume de données conséquent et désirant utiliser celles-ci pour anticiper le comportement de clients ou d'utilisateurs

de services. Nous avons vu lors du chapitre précédent la manière de structurer un problème en échantillons d'apprentissage et de validation (Figure 1) mais sans en expliquer la raison d'être.

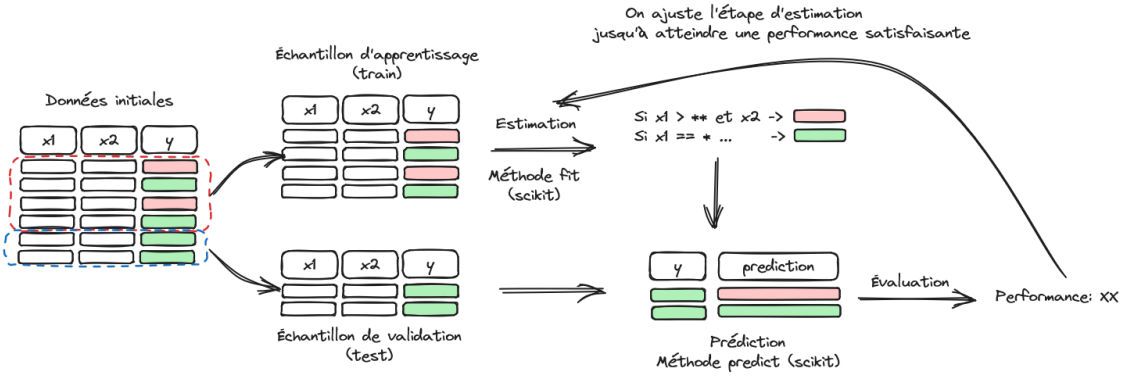


Figure 1. – Illustraton de la méthodologie du *machine learning*

1 Une méthodologie pour éviter le sur-apprentissage

Puisque l'objectif du *machine learning* est de mettre en oeuvre un modèle sur une population cible qui n'est pas celle sur laquelle le modèle a appris - par exemple un modèle de *scoring* ne sert pas à changer les prêts des clients existants mais à prendre une décision pour de nouveaux clients - il est logique de privilégier la validité externe d'un modèle. Pour que les anticipations de performance d'un modèle soient réalistes, il est donc nécessaire d'évaluer les modèles dans un cadre similaire à celui dans lequel il sera mis en oeuvre ultérieurement. Autrement dit, une évaluation honnête d'un modèle se doit d'être une évaluation de la validité externe d'un modèle, c'est-à-dire de la capacité à être bon sur une population qu'il n'a pas rencontré lors de son entraînement.

Pourquoi s'embarasser de cette considération ? Parce que construire un modèle sur un échantillon et l'évaluer sur celui-ci amène à une validité interne forte, au détriment de la validité externe. Autrement dit, si vous avez un contrôle sur les questions de cours, et strictement sur celles-ci, la meilleure stratégie à mettre en oeuvre est d'apprendre par coeur votre cours et le restituer comme tel. Un test de ce type ne cherche pas à savoir si vous comprenez votre cours, seulement si vous avez appris votre cours. Il s'agit de tester la validité interne de vos connaissances. Plus les questions s'éloigneront de ce que vous avez appris par coeur, plus vous serez en difficulté.

L'idée est la même pour un algorithme: plus son apprentissage est adhérant à un échantillon initial, plus ses performances prédictives, et donc sa valeur en pratique, seront limités. C'est pour cette raison qu'on évalue la qualité d'un modèle sur un échantillon qu'il n'a pas vu dans son apprentissage: pour privilégier la validité externe à la validité interne.

On parle de **sur-apprentissage** lorsqu'un modèle a une bonne validité interne mais une mauvaise validité externe, c'est-à-dire de mauvaises qualités prédictives sur un autre échantillon que celui sur lequel il a appris. La structuration d'un problème d'apprentissage sous la forme d'échantillons *train*/*test* est une réponse à ce défi puisqu'elle permet de sélectionner le meilleur modèle en extrapolation. Ce sujet peut apparaître trivial mais, dans les faits, de nombreux champs scientifiques empiriques n'adoptent pas cette méthodologie pour émettre des conclusions au-delà de la population sur laquelle ils ont travaillé.

Par exemple, en économie, il est assez commun d'évaluer une politique publique *toutes choses égales par ailleurs*, d'en déduire un effet marginal et de préconiser des recommandations politiques au-delà de celle-ci. Cependant, il est rare que la politique suivante s'applique à la même population

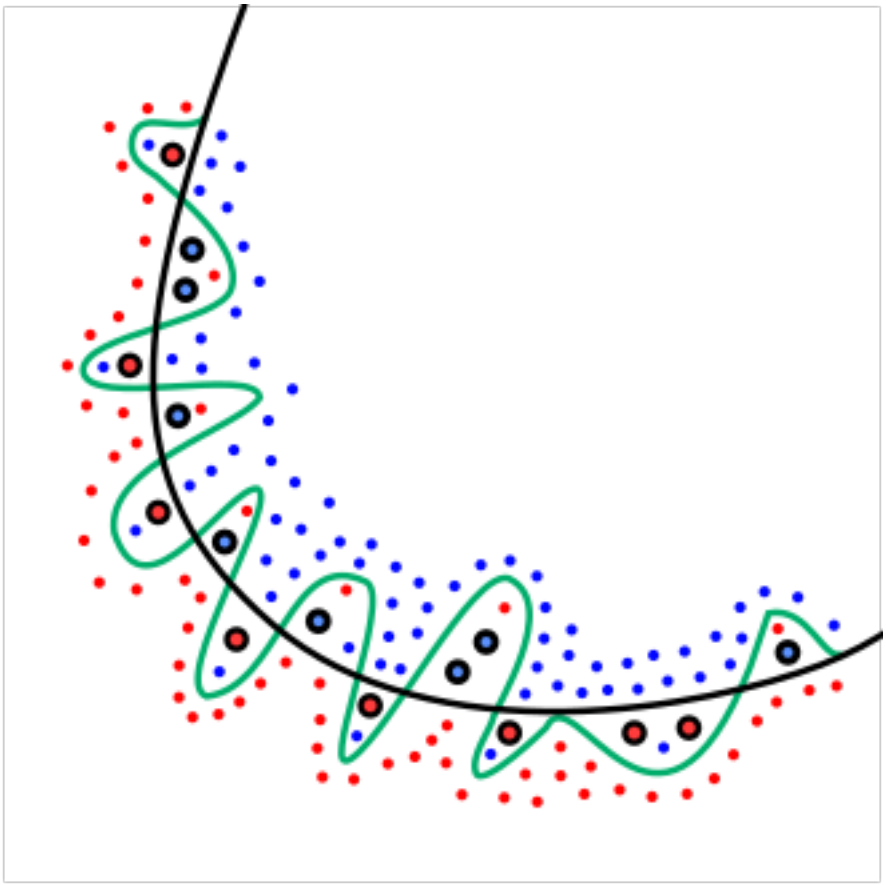
cible ou dans les mêmes conditions institutionnelles, de sorte qu'elle produit généralement des effets différents. Les biais d'échantillonnage, du point de vue des caractéristiques individuelles ou de la période d'étude, sont souvent relégués au second plan et l'estimation des effets marginaux se fait généralement indépendamment de toute notion de validité externe.

Pour en revenir à l'objet de ce chapitre, formellement, ce problème vient de l'**arbitrage biais-variance** dans la qualité d'estimation. Soit $h(X, \theta)$ un modèle statistique. On peut décomposer l'erreur d'estimation en deux parties :

$$\mathbb{E}\left[(y - h(\theta, X))^2\right] = \underbrace{\Big(y - \mathbb{E}(h_\theta(X))\Big)^2}_{\text{biais}^2} + \underbrace{\mathbb{V}\left(h(\theta, X)\right)}_{\text{variance}}$$

Il y a ainsi un compromis à faire entre biais et variance. Un modèle peu parcimonieux, c'est-à-dire proposant un grand nombre de paramètres, va, en général, avoir un faible biais mais une grande variance. En effet, le modèle va tendre à se souvenir d'une combinaison de paramètres à partir d'un grand nombre d'exemples sans être capable d'apprendre la règle qui permette de structurer les données.

Par exemple, la ligne verte ci-dessous est trop dépendante des données et risque de produire une erreur plus importante que la ligne noire (qui moyennise plus) sur de nouvelles données.



La division entre échantillons d'apprentissage et de validation est une première réponse au défi du surapprentissage. Néanmoins, ce n'est pas le seul geste méthodologique pour avoir un bon modèle prédictif.

De manière générale, il sera préférable d'adopter des modèles parcimonieux, c'est-à-dire les modèles faisant le moins d'hypothèses possibles sur la structure des données tout en offrant une performance satisfaisante. Ceci est généralement vu comme une illustration du principe du rasoir d'Ockham: en l'absence d'arguments théoriques, le meilleur modèle est celui qui permet d'expliquer au mieux les données avec les hypothèses les plus minimalistes. Cette approche, très opérationnelle, guidera de nombreux choix méthodologiques que nous mettrons en oeuvre.

2 Comment évaluer un modèle ?

L'introduction de cette partie présentait les principaux concepts pour se repérer dans la terminologie du *machine learning*. Si les concepts d'apprentissage supervisé, non supervisé, classification, régression, etc. ne sont pas clairs, il est recommandé de retourner voir ce chapitre. Pour rappel, le *machine learning* intervient dans les domaines où on ne dispose pas de modèles théoriques, en contrôlant tous les paramètres, faisant consensus et où on va chercher des règles statistiques, selon une démarche inductive. Ce n'est donc pas une approche scientifique justifiée dans tous les domaines. Par exemple, il vaut mieux privilégier le réglage des satelittes par le biais des équations de gravitation que par le biais d'un algorithme de *machine learning* qui risque d'introduire du bruit là où ce n'est pas nécessaire.

La principale ligne de partage entre les méthodes d'évaluation sera la nature du phénomène étudié (la variable y). Selon qu'on dispose ou non d'une mesure directe de notre variable d'intérêt, une sorte de *gold standard*, on mettra en oeuvre des métriques prédictives directes (cas de l'apprentissage supervisé) ou des métriques de stabilité statistique (apprentissage non supervisé).

Néanmoins, le succès des modèles de fondations, c'est-à-dire de modèles généralistes pouvant être utilisés pour des tâches auxquels ils n'ont pas été entraînés, amène à élargir la question de l'évaluation. Il n'est pas toujours évident de pouvoir définir l'objectif précis d'un modèle généraliste ni de pouvoir évaluer de manière consensuelle sa qualité. `ChatGPT` ou `Claude` nous apparaîssent bon, mais comment peut-on avoir une idée de leur pertinence dans différents cas d'usage ? Outre le sujet des annotations, ceci amène à réfléchir, plus globalement, sur le rôle de l'humain dans l'évaluation et le contrôle de décisions prises par des algorithmes.

2.1 Apprentissage supervisé

En apprentissage supervisé, on distingue en général les problèmes de:

- Classification : la variable y est discrète
- Régression : la variable y est continue

Les métriques mises en oeuvre peuvent être objectives dans ces deux cas car on dispose d'une valeur réelle, une valeur cible faisant office de *gold standard*, auquel comparer la valeur prédite.

2.1.a Classification

Le cas le plus simple à comprendre est celui de la classification binaire. Dans ce cas, soit on a juste, soit on se trompe, sans nuance.

La plupart des critères de performance consistent ainsi à explorer les différentes cases de la **matrice de confusion** :

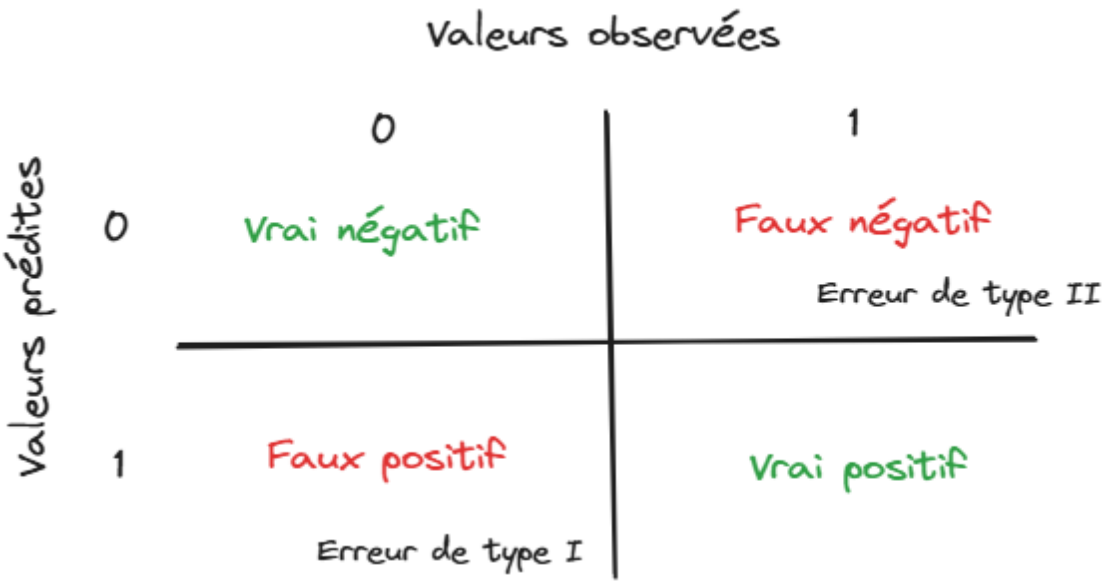


Figure 2. – Construction de la matrice de confusion

Cette dernière met en regard les valeurs prédites avec les valeurs observées. Le cas binaire est le plus simple à appréhender ; la classification multiclasse est une version généralisée de ce principe.

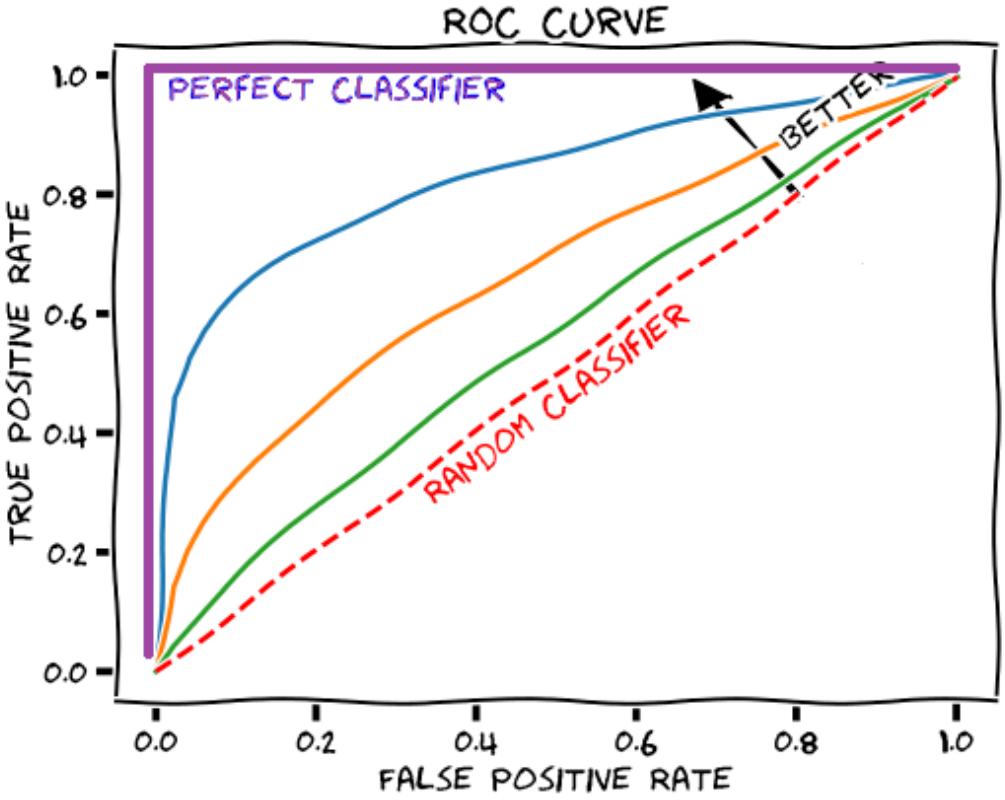
A partir des 4 coins de cette matrice, il existe plusieurs mesures de performance:

Critère	Mesure	Calcul
Accuracy	Taux de classification correcte	Diagonale du tableau : $\frac{TP+TN}{TP+FP+FN+FP}$
Precision	Taux de vrais positifs	Ligne des prédictions positives : $\frac{TP}{TP+FP}$
Recall (rappel)	Capacité à identifier les labels positifs	Colonne des prédictions positives : $\frac{TP}{TP+FN}$
F1 Score	Mesure synthétique (moyenne harmonique) de la précision et du rappel	$2\frac{precision \times recall}{precision+recall}$

Néanmoins, certaines métriques préfèrent plutôt prendre en compte les probabilités de prédiction. Si un modèle fait une prédiction mais avec une confiance très modérée et qu'on l'accepte, peut-on lui en tenir rigueur ? Pour cela, on fixe un seuil de probabilité c à partir duquel on prédit qu'une observation donnée appartient à une certaine classe prédite:

$$\mathbb{P}(y_i = 1|X_i) > c \Rightarrow \hat{y}_i = 1$$

Plus on augmente c, plus on est sélectif sur le critère d'appartenance à la classe.
La précision, i.e. le taux de vrais positifs parmi les prédictions positives, augmente. Mais on augmente le nombre de positifs manqués (ce sont des faux négatifs). Autrement dit, quand on est pointilleux, on diminue le rappel. Pour chaque valeur de c correspond une matrice de confusion et donc des mesures de performances. La **courbe ROC** consiste à faire varier c de 0 à 1 et vérifier l'effet sur les performances :



L'aire sous la courbe (**AUC**) permet d'évaluer quantitativement le meilleur modèle au sens de ce critère. L'AUC représente la probabilité que le modèle soit capable de distinguer entre la classe positive et négative.

2.1.b Régression

Lorsqu'on travaille sur une variable quantitative, l'objectif est d'avoir une prédiction la plus proche de celle-ci. Les indicateurs de performance en régression consistent donc à mesurer l'écart entre la valeur prédite et la valeur observée :

Nom	Formule
Mean squared error	$MSE = \mathbb{E}[(y - h_\theta(X))^2]$
Root Mean squared error	$RMSE = \sqrt{\mathbb{E}[(y - h_\theta(X))^2]}$
Mean Absolute Error	$MAE = \mathbb{E}\left[\lvert y - h_\theta(X)\rvert\right]$
Mean Absolute Percentage Error	$MAE = \mathbb{E}\left[\left\lvert\frac{y - h_\theta(X)}{y}\right\rvert\right]$

Ces métriques devraient rappeler des souvenirs si vous êtes familiers de la méthode des moindres carrés, ou plus généralement de la régression linéaire. Cette méthode vise justement à trouver des paramètres qui minimisent ces métriques, dans un cadre statistique formel.

2.2 Apprentissage non supervisé

Dans cet ensemble de méthodes, on ne dispose pas de *gold standard* pour comparer la prédiction à la valeur observée. Pour mesurer la performance d'un algorithme, il faut donc se résoudre à utiliser des métriques de stabilité des prédictions, basées sur des critères statistiques. Cela permet d'évaluer si le fait de complexifier l'algorithme change fondamentalement la distribution des prédictions.

Les métriques utilisées dépendent du type d'apprentissage mis en oeuvre. Par exemple, le *clustering* par *K-means* utilise généralement une mesure d'inertie qui quantifie l'homogénéité des clusters. Une

bonne performance correspond au cas où les clusters sont homogènes et se distinguent l'un de l'autre. Plus on a de clusters (le K de $K-means$), plus ceux-ci tendent à être homogènes. Si on ne choisit pas un K adéquat, on peut faire du sur-apprentissage: si on ne compare les modèles qu'en fonction de leur homogénéité, on va choisir un nombre de cluster très élevé ce qui correspond à un cas classique de surapprentissage. Les méthodes de sélection du nombre de cluster optimal, comme la méthode du coude, visent à évaluer le moment où le gain d'inertie lié à l'augmentation du nombre de clusters tend à s'affaisser. On sélectionne alors le nombre de clusters qui offre le meilleur compromis entre parcimonie et performance.

2.3 Comment sont évalués les grands modèles de langage et les IA génératives ?

S'il apparaît relativement intuitif d'évaluer des modèles supervisés (pour lesquels on dispose d'observations faisant office de vérité absolue), comment juger de la qualité d'un outil comme `ChatGPT` ou `Copilot` ? Comment définir une bonne IA générative : est-ce une IA qui fournit une information juste du premier coup (véracité) ? une IA qui fait preuve de capacité de raisonnements (*chain of thought*) dans une discussion ? Doit-on juger le style ou uniquement le fond ?

Ces interrogations sont des champs actifs de recherche. Les modèles de fondation étant très généraux, entraînés à différentes tâches, parfois de manière supervisée, parfois de manière non supervisée, il est difficile de définir un objectif unique permettant de considérer qu'un modèle est, sans ambiguité, meilleur qu'un autre. Le *leaderboard MTEB (Massive Text Embedding Benchmark)* présente par exemple de nombreuses métriques pour des tâches diverses et il peut être difficile de s'y retrouver. Sans compter que le rythme effrené de publication de nouveaux modèles change régulièrement ce classement.

Globalement, même s'il existe des métriques où on fait automatiquement évaluer la qualité d'un texte à un autre LLM (métriques *LLM as a judge*), pour avoir des modèles de langage de qualité, il est nécessaire d'avoir de l'évaluation humaine à plusieurs niveaux. En premier lieu, il est utile d'avoir un corpus de données annotées (des textes avec un résumé rédigé par un humain, des descriptions d'images, etc.) pour la phase d'entraînement et d'évaluation. Ceci permet de guider le comportement du modèle sur une tâche donnée.

Mais l'humain peut aussi intervenir *ex post*, pour faire remonter une évaluation de la qualité d'un modèle. On parle de *feedbacks*. Ceci peut prendre plusieurs formes, par exemple une évaluation positive ou négative de la réponse ou une évaluation plus qualitative. Ces informations remontées ne vont pas forcément servir à la version actuelle du modèle mais pourront servir ultérieurement pour entraîner un modèle selon une technique de renforcement.

2.4 Evaluer sans être tourné vers le passé: les enjeux de la supervision des modèles

Il convient de garder en mémoire qu'un modèle de *machine learning* est entraîné sur des données passées. Un usage opérationnel de celui-ci, dans la phase suivante de son cycle de vie, implique donc de faire des hypothèses fortes sur la stabilité des nouvelles données reçues. Si le contexte évolue, un modèle risque de ne plus apporter des performances satisfaisantes. Si dans certains cas cela peut être rapidement mesuré à partir d'indicateurs clés (ventes, nombre de nouveaux clients, etc.) il est tout de même important de conserver un contrôle sur les modèles.

Ceci ouvre à la notion d'**observabilité** d'un modèle de *machine learning*. En informatique, l'observabilité est le principe qui consiste à surveiller, mesurer et comprendre l'état d'une application pour s'assurer que celle-ci soit toujours en mesure de répondre à ses utilisateurs. L'idée de l'observabilité en *machine learning* est similaire : il s'agit de contrôler qu'un modèle permet toujours d'avoir des performances satisfaisantes au cours du temps. Le principal risque dans le cycle de vie

d'un modèle est celui du *data drift*, changement dans la distribution des données au fil du temps qui entraîne une dégradation des performances d'un modèle de *machine learning*. Bien sûr, avoir construit un modèle à bonne validité externe va réduire l'effet de ce risque mais il sera inévitable qu'il ait des effets si la structure des données change trop par rapport au contexte d'entraînement.

Pour garder un modèle pertinent dans le temps, il sera nécessaire de régulièrement collecter de nouvelles données (principe des annotations) et adopter une stratégie de ré-entraînement. Ceci ouvre la voie aux problématiques de la mise en production et du MLOps qui sont le point de départ d'un cours que Romain Avouac et moi donnons.

Informations additionnelles

ℹ Environnement Python

Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
    "ipykernel≥6.29.5",
    "jupyter≥1.1.1",
    "jupyter-cache≥1.0.0",
    "kaleido≥0.2.1",
    "langchain-community≥0.3.27",
    "loguru==0.7.3",
    "markdown≥3.8",
    "nbclient≥0.10.0",
    "nbformat≥5.10.4",
    "nltk≥3.9.1",
    "openpyxl≥3.1.5",
    "pandas≥3.0",
    "pip≥25.1.1",
    "plotly≥6.1.2",
    "plotnine≥0.15",
    "polars≥1.8.2",
    "py7zr≥1.1.3",
    "pyarrow≥17.0.0",
    "pynsee≥0.1.8",
```

```
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.